POPLUR

Poplur AI Backend Completion Guide

Step-by-step implementation plan for real backend endpoints, server-side Google AI Studio credentials, and Firestore transactional credit enforcement.

Prepared: 2026-07-04

What this document completes

- Premium Poplur AI requests are moved behind real backend endpoints.
- Google AI Studio runs server-side only through GOOGLE_AI_API_KEY.
- Credits are reserved, charged, and refunded in Firestore transactions.
- iOS and Android call the same authenticated API and never store provider secrets.

Milestone	Definition of done
Backend endpoints	Every AI action is exposed as an authenticated endpoint that creates an aiGenerationJob and returns a job id.
Secrets	GOOGLE_AI_API_KEY and any provider keys exist only in Firebase Secret Manager or the hosting platform secret store.
Credits	A Firestore transaction prevents negative balances and logs every spend, refund, grant, and radius boost.
Mobile	Both iOS and Android upload inputs, call backend endpoints, poll jobs, then publish generated assets to map/profile/channel.

1. Target Architecture

Use Firebase Cloud Functions v2 for the fastest path because Poplur already uses Firebase Auth and Firestore. Cloud Run also works, but the first complete version should minimize moving parts.

Mobile apps never call Google AI Studio directly.

iOS and Android send Firebase-authenticated requests to the backend. The backend validates the user, enforces credits transactionally, calls Google AI Studio with GOOGLE_AI_API_KEY, stores results, and returns a job id/status.

```
iOS / Android
-> Firebase Auth ID token
-> Backend HTTPS endpoint
-> Firestore transaction: reserve credits + create aiGenerationJob
-> Google AI Studio provider adapter reads GOOGLE_AI_API_KEY server-side
-> Firebase Storage stores output
-> Firestore transaction: complete job + charge or refund
-> App polls job and publishes to map/channel/profile
```

Non-negotiable rules

- Premium AI uses Google AI Studio through the backend only.
- Replicate is not the premium provider. Keep any free-tier fallback separate from premium routing.
- GOOGLE_AI_API_KEY is a server secret. It must never be checked into iOS, Android, Remote Config, or Firestore.
- Credits are never trusted from the client. The client can request work; the server decides if the user can pay.
- No job publishes to map/channel/profile until the backend confirms the asset belongs to that user and generation is complete.

Endpoint groups to build

Group	Endpoints
AI jobs	POST /ai/generate/image, /video, /edit, /vfx, /3d, /voice, /music, /game; GET /ai/jobs/:id; POST /ai/jobs/:id/cancel
Publishing	POST /ai/jobs/:id/publish-to-map; POST /posts/:id/publish-to-channel
Credits	GET /credits/balance; GET /credits/transactions
Radius boosts	POST /posts/:id/radius-boost
Business campaigns	POST /businesses/:businessId/ai-campaigns; GET /businesses/:businessId/ai-campaigns/:id

2. Backend Setup And Secrets

Create a backend workspace beside the apps, then initialize Firebase Functions with TypeScript.

```
mkdir backend
cd backend
firebase login
firebase use --add
firebase init functions firestore emulators

# Recommended choices
# Functions language: TypeScript
# Runtime: Node.js 20 or current Firebase-supported Node LTS
# Enable ESLint: yes if your team uses it
```

Install baseline dependencies from the functions directory.

```
cd backend/functions
npm install firebase-admin firebase-functions

# Add the current Google GenAI SDK or REST client according to Google AI docs.
# Keep model ids in config so model changes do not require mobile releases.
```

Add server-side secrets

Use Firebase Functions Secret Manager for GOOGLE_AI_API_KEY and any other provider key.

```
firebase functions:secrets:set GOOGLE_AI_API_KEY
firebase functions:secrets:set MESHY_API_KEY
firebase functions:secrets:set ELEVENLABS_API_KEY
firebase functions:secrets:set SUNO_API_KEY
firebase functions:secrets:set GAME_PROVIDER_API_KEY
firebase functions:secrets:set VFX_PROVIDER_API_KEY
```

Read the secret inside a function

```
import { onRequest } from "firebase-functions/v2/https";
import { defineSecret } from "firebase-functions/params";

const googleAIKey = defineSecret("GOOGLE_AI_API_KEY");

export const generateImage = onRequest(
{ secrets: [googleAIKey], timeoutSeconds: 300, memory: "1GiB" },
async (req, res) => {
const apiKey = googleAIKey.value();
// Use apiKey only inside the backend provider adapter.
}
);
```

Credential checklist

- Remove premium provider keys from iOS and Android source, plist, gradle files, and config assets.
- Do not expose GOOGLE_AI_API_KEY through Firebase Remote Config.
- Rotate any key that was previously shipped to devices.
- Restrict logs so prompts and provider raw responses do not leak private data.

3. Firestore Data Model

Create collections that separate balances, immutable ledger entries, generation jobs, generated assets, posts, boosts, and business campaigns.

Collection	Required fields
creatorCreditBalances/{uid}	availableCredits, reservedCredits, lifetimeEarned, lifetimeSpent, updatedAt, planTier
creatorCreditTransactions/{transactionId}	userId, amount, type, relatedJobId, relatedPostId, provider, contentType, createdAt, metadata
aiGenerationJobs/{jobId}	userId, provider, contentType, prompt, inputAssetIds, status, creditsReserved, creditsCharged, outputAssetIds, errorMessage, createdAt, updatedAt, completedAt
generatedAssets/{assetId}	userId, jobId, provider, contentType, storagePath, downloadUrl, mimeType, moderationStatus, createdAt
posts/{postId}	userId, assetIds, channelId, latitude, longitude, radiusMeters, organicRadiusMeters, boostedRadiusMeters, boostExpiresAt, visibilityStatus, businessId
radiusBoostTransactions/{id}	userId, postId, amount, radiusMetersAdded, expiresAt, createdAt
aiCampaigns/{id}	businessId, ownerUid, prompt, assetIds, status, channels, coupon, caption, tags, createdAt, updatedAt

Status values

aiGenerationJobs.status

- queued
- processing
- provider_pending
- completed
- failed
- cancelled
- refunded

moderationStatus

- not_required
- pending
- approved
- rejected
- needs_review

Ledger principle

Treat creatorCreditTransactions as the audit trail. Balances are fast summaries, but every balance change must have a matching transaction row. That makes refunds, support investigations, and fraud checks possible.

4. Auth And Security Rules

All write paths that affect money, credits, AI jobs, generated assets, or boosted reach should go through the backend. Mobile clients can read their own records but cannot directly mutate balances or job status.

Backend auth middleware

Every endpoint should require a Firebase ID token in the Authorization header.

```
Authorization: Bearer <firebase-id-token>

Backend steps:
1. Read the Bearer token.
2. admin.auth().verifyIdToken(token).
3. Use decoded.uid as the only trusted user id.
4. Reject any body.userId that does not match decoded.uid.
```

Minimum Firestore rules posture

```
match /creatorCreditBalances/{uid} {
allow read: if request.auth.uid == uid;
allow write: if false;
}

match /creatorCreditTransactions/{id} {
allow read: if request.auth.uid == resource.data.userId;
allow write: if false;
}

match /aiGenerationJobs/{jobId} {
allow read: if request.auth.uid == resource.data.userId;
allow write: if false;
}

match /generatedAssets/{assetId} {
allow read: if request.auth.uid == resource.data.userId;
allow write: if false;
}
```

Why writes are blocked

The Admin SDK used by Cloud Functions bypasses client rules. That is good here: it lets backend code write trusted records while stopping clients from faking completed jobs, changing credit balances, or publishing someone else's generated asset.

5. Atomic Credit Enforcement

The credit system should reserve credits before the provider call, then either charge the reserved amount on success or refund it on failure. The reserve step must happen inside a Firestore transaction.

Step	Server action
1. Estimate	Calculate cost from contentType, provider, duration, resolution, and planTier using a server-side cost table.
2. Transaction	Read creatorCreditBalances/{uid}; verify availableCredits >= cost; update availableCredits -= cost and reservedCredits += cost.
3. Ledger	Create creatorCreditTransactions row with type ai_generation_reserve, amount -cost, relatedJobId.
4. Job	Create aiGenerationJobs/{jobId} with status processing and creditsReserved = cost.
5. Provider	Call Google AI Studio or another server-side provider adapter.
6A. Success	Create generatedAssets, update job completed, move reservedCredits down, increment lifetimeSpent.
6B. Failure	Update job failed, add refund transaction, restore availableCredits, decrease reservedCredits.

Transaction pseudocode

```
await db.runTransaction(async (tx) => {
const balanceRef = db.doc(`creatorCreditBalances/${uid}`);
const balanceSnap = await tx.get(balanceRef);
const balance = balanceSnap.data() ?? { availableCredits: 0, reservedCredits: 0 };

if (balance.availableCredits < cost) {
throw new InsufficientCreditsError();
}

tx.update(balanceRef, {
availableCredits: balance.availableCredits - cost,
reservedCredits: balance.reservedCredits + cost,
updatedAt: FieldValue.serverTimestamp(),
});

tx.create(jobRef, {
userId: uid,
status: "processing",
creditsReserved: cost,
contentType,
provider,
createdAt: FieldValue.serverTimestamp(),
});

tx.create(transactionRef, {
userId: uid,
amount: -cost,
type: "ai_generation_reserve",
relatedJobId: jobRef.id,
createdAt: FieldValue.serverTimestamp(),
});
});
```

6. Implement Endpoints In This Order

Build the smallest complete vertical slice first: image generation with real credits, real job record, real provider call, storage output, and mobile polling. Then copy the same pattern to video, voice, music, 3D, games, VFX, and business campaigns.

1. GET /credits/balance - authenticated, read-only balance summary.
2. GET /ai/jobs/:id - authenticated job lookup; only owner can read.
3. POST /ai/generate/image - reserve credits, call Google AI Studio, store output, complete/refund.
4. POST /ai/generate/video - same contract as image, but allow provider_pending for longer-running jobs.
5. POST /ai/generate/edit - accepts source asset id plus prompt; verifies ownership before provider call.
6. POST /ai/generate/vfx - adds cinematic effects to an existing asset.
7. POST /ai/generate/3d - generates or updates 3D head/avatar/asset.
8. POST /ai/generate/voice - creates narration using configured provider.
9. POST /ai/generate/music - creates soundtrack or short loop.
10. POST /ai/generate/game - creates mini-game concept/assets for business campaigns.
11. POST /ai/jobs/:id/cancel - cancels cancellable jobs and refunds any reserved amount.
12. POST /ai/jobs/:id/publish-to-map - publishes completed generated asset with map location.
13. POST /posts/:id/radius-boost - deducts credits and boosts radius atomically.
14. POST /businesses/:businessId/ai-campaigns - builds a campaign bundle for verified business accounts.

Shared endpoint response

```
{
"jobId": "abc123",
"status": "processing",
"provider": "google_ai_studio",
"contentType": "image",
"creditsReserved": 12,
"estimatedSeconds": 30
}
```

Queue rule

If an operation can exceed a normal HTTPS timeout, return quickly after creating the job and move provider work to Cloud Tasks, Pub/Sub, or a background queue. The mobile app should already be polling job status, so the UX stays the same.

7. Google AI Studio Provider Adapter

Wrap Google AI Studio behind a provider adapter so Poplur can swap or route providers without changing iOS and Android again.

Adapter interface

```
export interface AIProviderAdapter {
generateImage(input: GenerateInput): Promise<GenerateResult>;
generateVideo(input: GenerateInput): Promise<GenerateResult>;
editAsset(input: EditInput): Promise<GenerateResult>;
}

export type GenerateResult = {
provider: "google_ai_studio";
contentType: "image" | "video";
mimeType: string;
bytes?: Buffer;
remoteUrl?: string;
providerJobId?: string;
raw?: unknown; // store admin-only or omit from client responses
};
```

Implementation steps

1. Read GOOGLE_AI_API_KEY inside the Cloud Function using defineSecret or your host's secret API.
2. Build prompt payload from the user's single Poplur AI prompt plus server-side safety/context fields.
3. Call the selected Google model for image or video based on contentType.
4. Normalize the response into GenerateResult.
5. Store bytes or downloaded media in Firebase Storage under generated/{uid}/{jobId}/output.
6. Create generatedAssets record with storagePath, contentType, provider, and moderationStatus.
7. Return only jobId/status to mobile; never return provider secrets or full raw provider responses.

Provider routing

Keep a server-side route table: premium routes to google_ai_studio; free-tier routes can use your cheaper/free experience. The mobile app should pass intent, not provider names, unless you intentionally expose provider selection.

8. iOS And Android Hookup

Both platforms should share the same backend contract. Mobile gathers media, prompt, location, and channel selection; backend decides credits, provider, and publish validity.

iOS steps

1. Create PoplurAIBackendClient.
2. Read base URL from environment/config.
3. Get Firebase Auth currentUser ID token.
4. Send Authorization: Bearer token.
5. Upload source media to signed upload URL or Firebase S
6. Call /ai/generate/image or /video.
7. Poll /ai/jobs/:id until completed/failed.
8. Show output preview and call publish endpoint.

Android steps

1. Create PoplurAIBackendClient.
2. Use FirebaseAuth.getInstance().currentUser.
3. Fetch ID token before each protected request.
4. Send the same JSON body as iOS.
5. Use WorkManager/coroutines for polling.
6. Render completed output from generatedAssets URL.
7. Call publish endpoint with channel and map location.
8. Do not store provider keys in BuildConfig.

Generate request body

```
{
"contentType": "video",
"prompt": "Make this look like a movie trailer for the restaurant",
"sourceAssetId": "optional-existing-upload",
"location": { "lat": 44.3567, "lng": -79.6418 },
"channelId": "food",
"postIntent": "premium_poplur_ai"
}
```

UX contract

The app should show one Poplur AI prompt textarea, submit it, then show a job progress state. If generation fails, show the server error and confirm credits were refunded when applicable.

9. Publish, Radius Boost, And Business Campaigns

Publish generated asset to the map

1. Verify Firebase ID token and job ownership.
2. Read aiGenerationJobs/{jobId}; require status completed.
3. Verify every generatedAssets id belongs to the same uid/job.
4. Validate latitude, longitude, selected channel, and radius.
5. Run content/channel classification. If the selected channel is wrong, record originalChannelId and chosenChannelId.
6. Create posts/{postId} with map, profile, channel, and generated asset references.
7. Return postId and chosenChannelId to both mobile platforms.

Radius boost transaction

Radius boosts are just another credit spend and must use the same ledger approach.

```
POST /posts/:id/radius-boost
{
"tier": "local_plus",
"radiusMetersAdded": 5000,
"durationHours": 24
}

Server transaction:
- verify owner or business admin
- calculate cost
- deduct credits
- create radiusBoostTransactions row
- update post boostedRadiusMeters and boostExpiresAt
```

Business campaign generator

For verified businesses, the prompt can create a bundle: generated ad image/video, caption, coupon, tags, target channel, radius recommendation, and optional voice/music/game assets.

Campaign field	Purpose
businessId / ownerUid	Attach campaign to verified business profile and admins.
prompt	Single business intent prompt.
assetIds	Generated image/video/audio/game assets.
coupon	Optional discount or CTA generated by AI.
channels	AI-recommended channels plus user requested channel.
status	draft, generating, ready, published, failed.

10. Tests, Deploy, And Done Checklist

Automated tests to add first

- Free user without credits cannot create premium generation job.
- User with enough credits reserves credits and creates aiGenerationJobs row.
- Provider success completes job, stores generatedAssets, and charges reserved credits.
- Provider failure refunds credits and writes failed job with errorMessage.
- Two simultaneous requests cannot drive availableCredits below zero.
- Mobile cannot write creatorCreditBalances or aiGenerationJobs directly.
- Publish endpoint rejects jobs owned by another user.
- Channel classifier can override an inappropriate selected channel and records both channel ids.
- Radius boost deducts credits and updates post radius atomically.

Local and production deploy path

```
cd backend/functions
npm test
npm run lint

cd ..
firebase emulators:start

# After emulator QA from iOS and Android:
firebase deploy --only functions,firestore:rules,firestore:indexes
```

Feature complete means

- GOOGLE_AI_API_KEY exists server-side and is never present in app bundles.
- iOS and Android can generate image/video through backend and poll job status.
- Credits reserve, charge, refund, and radius boost through Firestore transactions.
- Generated content can be published to map/channel/profile after backend validation.
- Business campaign endpoint exists for verified business accounts.
- Rules block client writes to balances, ledger, jobs, and generated assets.

Official references

- Firebase HTTPS Functions: https://firebase.google.com/docs/functions/http-events
- Firebase Functions secrets and environment config: https://firebase.google.com/docs/functions/config-env
- Firebase Admin verify ID tokens: https://firebase.google.com/docs/auth/admin/verify-id-tokens
- Cloud Firestore transactions: https://firebase.google.com/docs/firestore/manage-data/transactions
- Google AI Studio / Gemini API key docs: https://ai.google.dev/gemini-api/docs/api-key
- Gemini API documentation: https://ai.google.dev/gemini-api/docs